

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ
ФЕДЕРАЦИИ
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ ОБРАЗОВАТЕЛЬНОЕ
УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
«НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ ТЕХНОЛОГИЧЕСКИЙ
УНИВЕРСИТЕТ «МИСИС»

ИНСТИТУТ Информационных компьютерных наук (ИКН)
КАФЕДРА Автоматизированных систем управления (АСУ)
НАПРАВЛЕНИЕ 09.03.01 ГРУППА БИВТ-24-8

КУРСОВОЙ ПРОЕКТ

ПО КУРСУ Клиент-серверных приложений и сетевых технологий.
ТЕМА: Разработка клиент-серверного приложения. Детали для ПК.

Студент Бу Тхань Бинь

Руководитель проекта Микитенко Игорь Иванович

Оценка выполнения курсового проекта _____

Москва 2026 г.

СОДЕРЖАНИЕ

1. Предметная область	4
2. Постановка задачи.....	4
3. Описание архитектуры приложения	5
4. Описание структуры базы данных	7
5. Описание серверной части (backend)	8
6. Описание клиента.....	10
ЗАКЛЮЧЕНИЕ	14
БИБЛИОГРАФИЧЕСКИЙ СПИСОК.....	15
ПРИЛОЖЕНИЕ А: Полный DDL-скрипт развертывания реляционной базы данных	16
ПРИЛОЖЕНИЕ Б: Исходный код серверных компонентов NestJS (Бизнес-логика).....	19
ПРИЛОЖЕНИЕ В: Исходный код клиентских компонентов React (Пользовательский интерфейс)	22

ВВЕДЕНИЕ

В настоящее время информационные технологии активно используются для автоматизации бизнес-процессов в различных отраслях. Важной задачей является организация эффективного корпоративного управления и распределения ресурсов, поскольку использование автоматизированных систем помогает повысить производительность ИТ, снизить количество ошибок при транзакциях и обеспечить оперативный доступ к аналитике в режиме реального времени.

Эта тема актуальна, поскольку в современных высокотехнологичных компаниях объем управленческих данных быстро превышает возможности традиционных методов контроля. Отсутствие единой ИТ-среды приводит к высокому риску ошибок, несогласованности бюджетов подразделений, срыву сроков проектов и отсутствию прозрачности в управлении задачами. Разработанное решение является отличным инструментом для руководителей и менеджеров, которым необходимо быстро проверять статус сотрудников и назначенные им задачи. Внедрение клиент-серверного веб-приложения позволяет централизованно хранить данные, использовать гибридную обработку в оперативной памяти для минимизации задержек интерфейса, автоматизировать каскадные обновления бизнес-показателей и строго разграничивать права доступа.

Цель данной курсовой работы — разработка распределенного клиент-серверного приложения «OrgSphere» для цифровизации процессов корпоративного управления, учета персонала и управления проектами с использованием фреймворка NestJS, библиотеки React и базы данных PostgreSQL.

1. Провести системный анализ предметной области и разработать структуру реляционной базы данных PostgreSQL;
2. Реализовать клиентское приложение в виде SPA-интерфейса на основе библиотеки React;
3. Разработать серверное приложение с использованием фреймворка NestJS и TypeScript;
4. Организовать взаимодействие между клиентской и серверной сторонами посредством асинхронных HTTP-запросов через Axios;
5. Создать программируемые объекты на стороне СУБД (представления, хранимые процедуры и триггеры) для автоматического управления бизнес-логикой;
6. Реализовать гибридную модель данных с кэшированием реестра пользователей в оперативной памяти и уровнем персистентности TypeORM;
7. Обеспечить строгую проверку DTO входящих данных и глобальную обработку ошибок;
8. Провести всестороннее функциональное тестирование разработанного программного обеспечения.

1. Предметная область

Курсовая работа посвящена автоматизации внутренних процессов в современной ИТ-компании. Система «OrgSphere» предназначена для управления организационной структурой, ведения кадрового учета и контроля выполнения проектных задач. В рамках приложения отслеживается деятельность отделов, каждый из которых имеет свой бюджет, а также хранится информация о сотрудниках (их имена, возраст, роли и контакты). Для руководства и менеджеров система служит удобным инструментом, позволяющим в любой момент проверить текущую нагрузку сотрудников и узнать, в каком статусе находятся их задачи. Вся информация хранится в единой базе данных PostgreSQL, а права на добавление и редактирование записей строго разграничены и доступны только пользователям с правами администратора.

2. Постановка задачи

Разрабатываемая система «OrgSphere» предназначена для автоматизации процессов корпоративного менеджмента, учета персонала, распределения ресурсов и контроля выполнения проектных задач ИТ-предприятия, а также для организации взаимодействия пользователей с системой посредством веб-интерфейса.

Основной задачей приложения является обеспечение удобного доступа к данным о сотрудниках, отделах и текущих задачах, а также предоставление средств управления этими данными в зависимости от роли пользователя. Система служит отличным инструментом для руководства, когда необходимо оперативно проверить текущий рабочий статус сотрудников и ход выполнения закрепленных за ними проектов.

В рамках выполнения курсовой работы необходимо реализовать следующие функциональные возможности:

- Разграничение прав доступа между администратором и обычными сотрудниками (ролевая модель);
- Полную поддержку CRUD-операций (просмотр, создание, редактирование и увольнение) для сущностей «Сотрудники» и «Департаменты»;
- Просмотр, добавление и управление Проектами и Задачами компании;
- Автоматический перевод сотрудника между отделами в рамках одной транзакции базы данных;
- Реактивный триггерный механизм для каскадного закрытия проекта при завершении всех его задач;
- Триггер безопасности для валидации возраста сотрудников (не младше 18 лет) и триггер финансового аудита для логирования изменений бюджетов департаментов;
- Создание реляционного представления (View) для быстрого сбора аналитики по текущей нагрузке персонала;
- Организацию гибридной модели данных с in-memory кэшированием реестра пользователей на сервере и синхронизацией с персистентным хранилищем;
- Строгую DTO-валидацию входных параметров и глобальную обработку ошибок на всех уровнях приложения.

При разработке приложения использовались следующие программные средства и технологии:

- Язык разметки HTML и каскадные таблицы стилей CSS;
- Библиотека React для построения пользовательского интерфейса (Frontend);

- Сетевой клиент Axios для выполнения асинхронных HTTP-запросов;
- Фреймворк NestJS и язык программирования TypeScript для реализации серверной части (Backend);
- Объектно-реляционный маппер TypeORM для взаимодействия с базой данных;
- Система управления базами данных PostgreSQL и встроенный язык PL/pgSQL для программирования логики на стороне ядра БД;
- Библиотека class-validator для проверки корректности вводимых данных.

3. Описание архитектуры приложения

Архитектура корпоративного портала «OrgSphere» построена по принципу разделения клиентской и серверной частей (Decoupled Architecture). Клиентское приложение на React и серверная часть на NestJS работают как полностью независимые модули. Они запущены в отдельных процессах и общаются друг с другом исключительно через сеть с помощью стандартного REST API. Это делает систему гибкой, так как изменение интерфейса никак не ломает внутреннюю логику сервера.

Взаимодействие между слоями приложения происходит по четкой цепочке:

- Интерфейс пользователя (React): работает прямо в браузере. Когда менеджер совершает какое-то действие — например, хочет проверить, в каком статусе находятся сотрудники, или добавляет новую задачу — React запускает асинхронную функцию. Она отправляет на сервер стандартный HTTP-запрос (GET, POST, PUT или DELETE) с данными в формате JSON.
- Контроллеры (NestJS): принимают этот запрос на стороне сервера. Перед тем как начать обработку, фреймворк пропускает данные через валидационные пайпы (ValidationPipe). Если клиент прислал пустую строку, текст вместо числа или нарушил другие правила, сервер сразу же отклоняет запрос с ошибкой 400 Bad Request, не тратя свои ресурсы.
- Сервисы (NestJS): если данные верны, контроллер передает их в слой бизнес-логики. Сервисы содержат все правила работы компании и спроектированы так, чтобы не зависеть от конкретной базы данных напрямую — они работают через абстрактные репозитории.

Особенностью «OrgSphere» является гибридная модель хранения данных, которая сочетает работу в оперативной памяти (In-Memory) и надежную запись на диск. Это сделано для демонстрации различных подходов к организации архитектуры веб-приложений.

Для сущности сотрудников на стороне бэкенда создан простой приватный массив:

TypeScript

```
private usersMemoryRepository: User[] = [];
```

Когда клиент запрашивает или обновляет данные пользователей, сервер в первую очередь работает с этим массивом в оперативной памяти. Это обеспечивает моментальный отклик интерфейса (Low Latency), так как приложению не нужно каждый раз тратить время на долгие запросы к диску или сетевые задержки при связи с базой данных. Такой подход наглядно моделирует работу систем кэширования в реальных высоконагруженных проектах.

При этом для надежного и долгосрочного хранения всех данных (включая отделы, проекты и задачи) используется полноценная реляционная база данных PostgreSQL. Синхронизацией объектов из кода и таблиц на диске занимается ORM-слой TypeORM.

Механизм их совместной работы устроен следующим образом: когда создается новый пользователь, NestJS добавляет его в in-мемори массив и сразу же отправляет команду на запись в PostgreSQL через TypeORM. Если база данных отклоняет операцию (например, если сработал триггер проверки возраста или email уже занят), TypeORM выдает ошибку. Сервер перехватывает это исключение, автоматически удаляет некорректного пользователя из оперативной памяти (происходит откат состояния) и возвращает клиенту понятное сообщение об ошибке. Благодаря этому данные в кэше сервера и в физической базе данных всегда остаются согласованными.

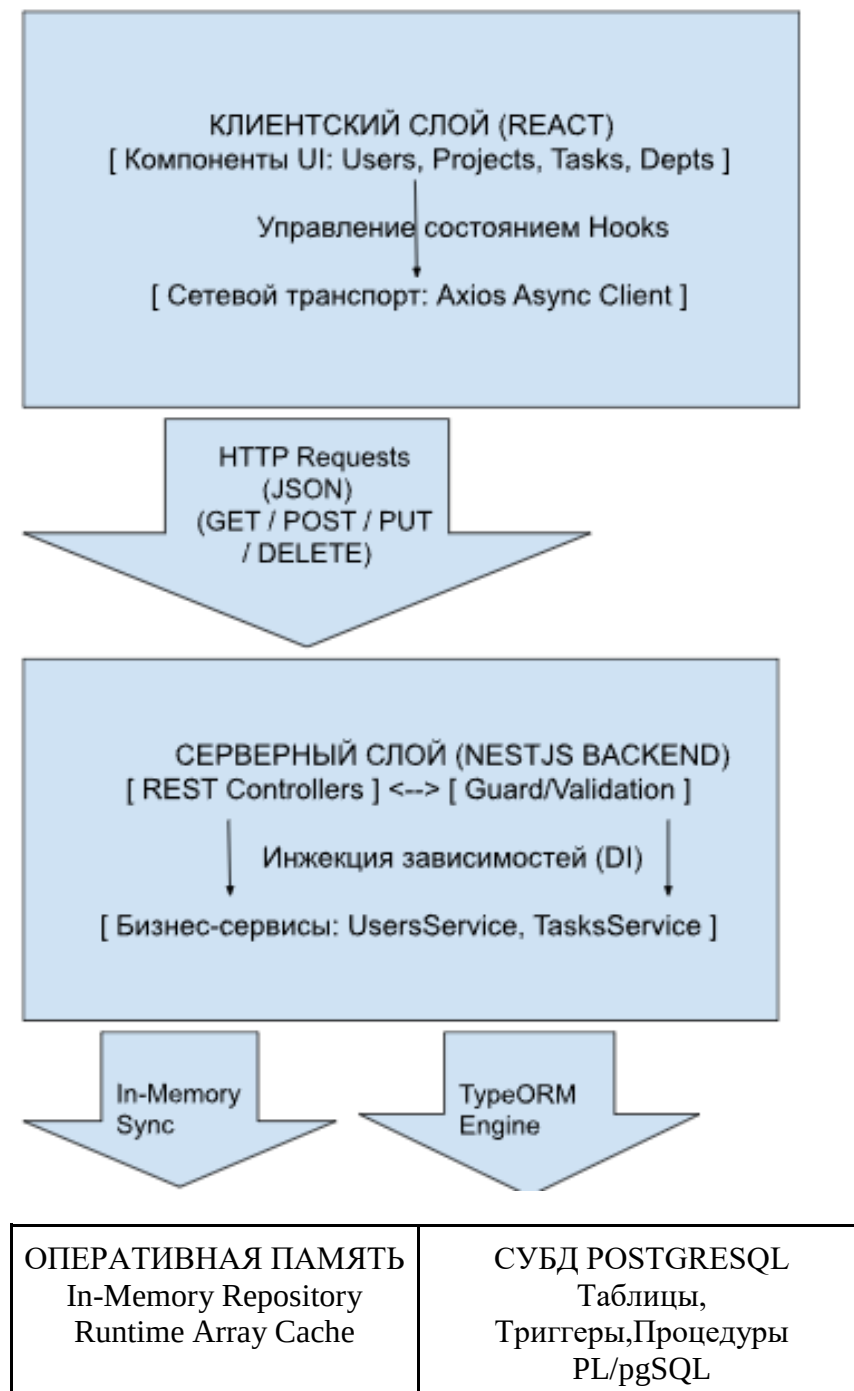


Рисунок 1. Представление конструкции приложения

4. Описание структуры базы данных

В разработанном клиент-серверном приложении «OrgSphere» для хранения данных используется реляционная система управления базами данных PostgreSQL.

База данных предназначена для хранения информации о сотрудниках, подразделениях (департаментах), проектах и закрепленных за ними задачах. Структура базы данных построена таким образом, чтобы исключить избыточное дублирование информации (все таблицы приведены к третьей нормальной форме) и обеспечить логические связи между основными сущностями компании.

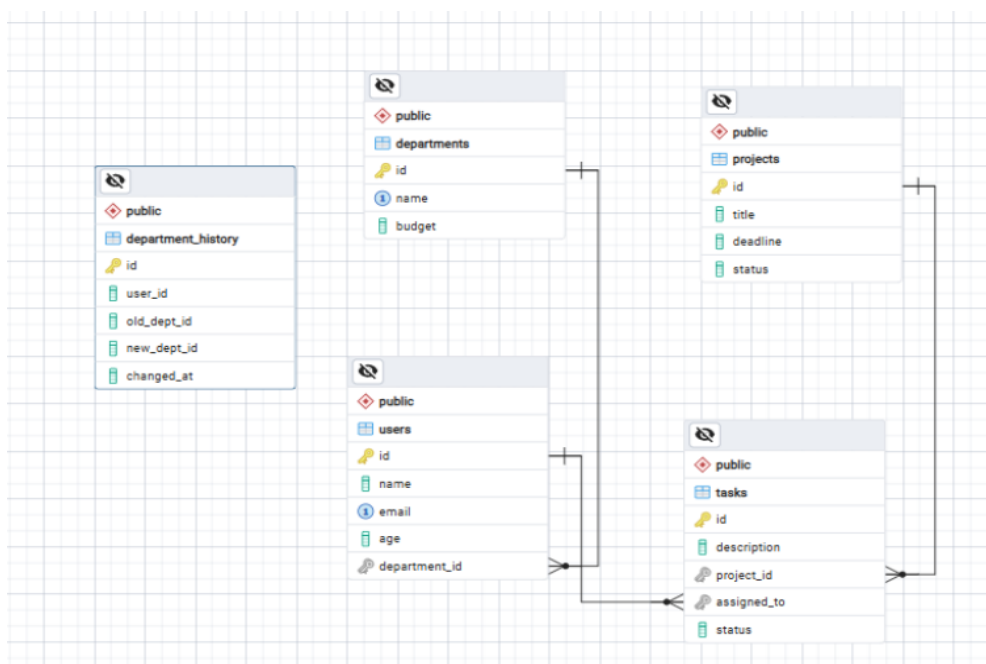


Рисунок 2. Схема базы данных приложения

В базе данных реализованы следующие основные таблицы:

- departments;
- users;
- projects;
- tasks.

Таблица **departments** предназначена для хранения информации о структурных подразделениях ИТ-предприятия. Каждый отдел является центром финансовой ответственности и имеет фиксированный бюджет. Состав полей таблицы departments:

- `id` - уникальный идентификатор департамента, первичный ключ;
- `name` - название подразделения;
- `budget` - финансовый бюджет отдела.

Таблица **users** хранит анкетные и системные данные персонала организации. Каждый сотрудник обязательно привязан к одному из департаментов (связь «один-ко-многим»). Удаление отдела заблокировано, если в нем числятся активные сотрудники. Кроме того, это отличный инструмент для руководителей, позволяющий в любой момент проверить, в каком статусе находятся сотрудники и кто за что отвечает. Состав полей таблицы users:

- id - уникальный идентификатор сотрудника, первичный ключ;
- name - имя и фамилия сотрудника;
- email - адрес электронной почты;
- age - возраст сотрудника;
- role_id - идентификатор роли пользователя в системе;
- department_id - идентификатор департамента, к которому привязан сотрудник (внешний ключ).

Таблица **projects** содержит сведения о текущих комплексных разработках компании. Каждый проект ограничен по времени и имеет определенное состояние. Состав полей таблицы projects:

- id - уникальный идентификатор проекта, первичный ключ;
- title - название проекта;
- deadline - дата окончания проекта (дедлайн);
- status - текущий статус проекта («Pending», «In Progress», «Completed»).

Таблица **tasks** предназначена для ведения конкретных поручений внутри проектов. Каждая задача связана с родительским проектом (при удалении проекта все его задачи удаляются каскадно) и закрепляется за ответственным сотрудником. Если сотрудник удаляется из системы, поле исполнителя сбрасывается в пустое значение (NULL). Состав полей таблицы tasks:

- id - уникальный идентификатор задачи, первичный ключ;
- description - текстовое описание сути задачи;
- project_id - идентификатор связанного проекта (внешний ключ);
- assigned_to - идентификатор сотрудника-исполнителя (внешний ключ);
- status - статус выполнения задачи.

5. Описание серверной части (backend)

APPMODULE (Инициализация TypeOrmModule.forRoot() и глобальная сборка DI)			
---	--	--	--

USERMODULE	DEPARTMODULE	PROJECTMODULE	TASKMODULE
UserController	DeptController	ProjController	TaskController
UsersService	DeptsService	ProjService	TasksService

Рисунок 3. серверная система

Серверная часть разработанного приложения является основным компонентом системы, обеспечивающим обработку пользовательских запросов, взаимодействие с базой данных, валидацию информации и разграничение прав доступа. В рамках данной курсовой работы

серверная часть реализована на базе фреймворка NestJS и языка программирования TypeScript, который обеспечивает высокую надежность, типизацию данных и модульную структуру приложения.

Основной задачей серверной части является организация четкого взаимодействия между пользовательским интерфейсом на React и базой данных PostgreSQL. Когда менеджер работает с порталом через браузер (например, проверяет текущие статусы сотрудников, добавляет новые задачи или перераспределяет бюджеты), все его действия передаются на бэкенд, где они обрабатываются, после чего результат возвращается клиенту.

Архитектура бэкенда спроектирована по модульному принципу. Вся логика разделена на четыре основных изолированных модуля: User Module (управление персоналом), DepartModule (управление отделами и финансами), ProjectModule (управление проектами) и TaskModule (работа с задачами). Главным элементом бэкенда является корневой модуль AppModule, который собирает все части приложения воедино и отвечает за глобальное подключение к СУБД PostgreSQL через TypeORM.

Внутри каждого модуля разработаны два ключевых компонента:

- Контроллеры (Controllers): файлы, которые отвечают за прием входящих HTTP-запросов и маршрутизацию. Например, UserController принимает запросы, связанные с персоналом. Они не содержат сложной логики, а лишь передают данные дальше.
- Сервисы (Services): файлы, где сосредоточена вся бизнес-логика и правила работы компании. Сервисы выполняют расчеты, обращаются к базе данных и вызывают хранимые процедуры.

Для взаимодействия между клиентской и серверной частями используются асинхронные HTTP-запросы в формате JSON. Получение информации (например, вывод аналитики по нагрузке) выполняется методом GET. Создание новых записей осуществляется методом POST. Изменение существующих данных (перевод сотрудника или обновление статуса задачи) выполняется методом PUT, а удаление записей — методом DELETE. Это соответствует принципам REST API и обеспечивает единообразный способ работы с системой.

Важным элементом архитектуры является гибридное управление данными в UsersService. Для демонстрации различных подходов, базовая работа с пользователями дублируется в оперативной памяти сервера (In-Memory) в виде локального массива, что обеспечивает мгновенный отклик API, но при этом все долгосрочные изменения синхронизируются с физическими таблицами PostgreSQL через слой TypeORM.

Перед выполнением любых операций изменения данных сервер осуществляет жесткую проверку входной информации. Для этого используются специальные классы DTO (Data Transfer Object) и библиотека class-validator. На входе в приложение автоматически проверяется формат электронной почты сотрудников, отсутствие пустых полей, а также контролируется возраст (в систему нельзя добавить пользователя младше 18 лет). Любые некорректные запросы или лишние поля, присланные в JSON-пакете, отсекаются еще до того, как они попадут в базу данных, а клиент получает понятное сообщение об ошибке.

Для обработки исключительных ситуаций на бэкенде развернуты глобальные фильтры исключений (Exception Filters). Все основные операции взаимодействия с СУБД защищены. Если в базе данных происходит сбой (например, срабатывает триггер на ограничение возраста или нарушается уникальность email), система перехватывает низкоуровневое

исключение, логирует его для администраторов, а пользователю на фронтенд возвращает структурированный и понятный JSON-ответ с описанием проблемы. Это предотвращает утечку служебной информации и делает приложение устойчивым к сбоям.

Таким образом, серверная часть приложения полностью обеспечивает выполнение всех функций автоматизации корпоративного менеджмента «OrgSphere». Она берет на себя маршрутизацию HTTP-запросов, разграничивает права доступа с помощью ролевой модели, контролирует корректность данных, взаимодействует с PostgreSQL и предоставляет готовый программный интерфейс API для клиентской стороны.

6. Описание клиента

Клиентская часть приложения OrgSphere разработана для обеспечения взаимодействия пользователя с системой через графический веб-интерфейс. В отличие от традиционных многостраничных веб-сайтов, фронтенд реализован как одностраничное приложение (SPA) на основе библиотеки React, что обеспечивает мгновенный отклик интерфейса и плавную, живую перерисовку элементов без перезагрузки страницы.

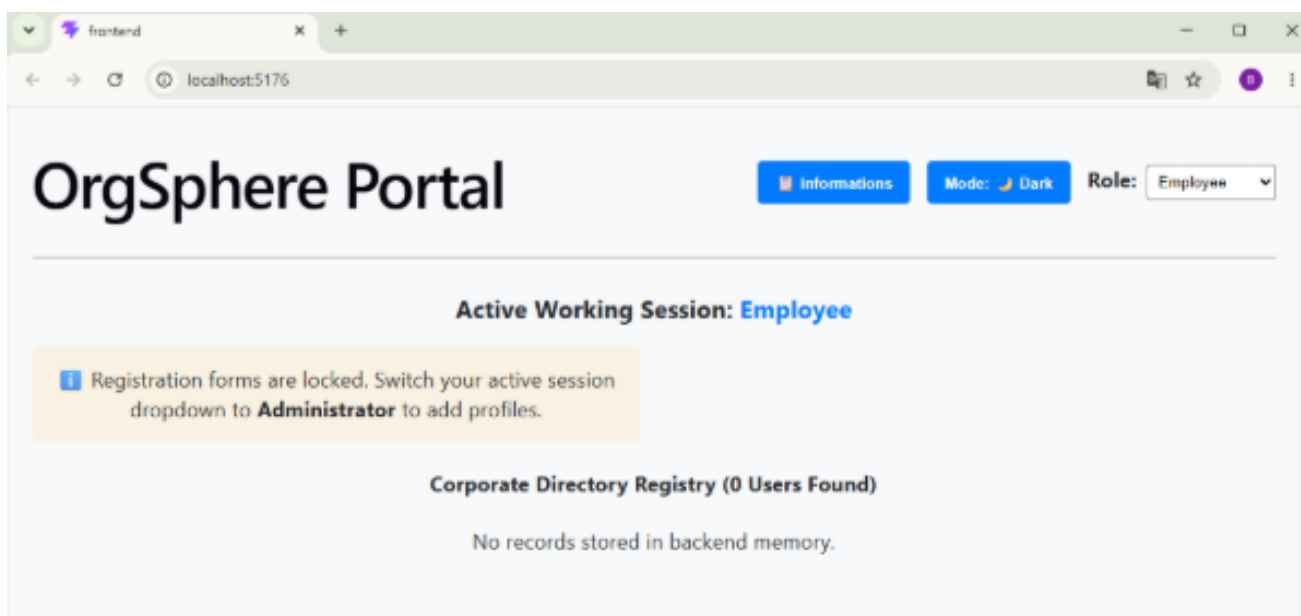


Рисунок 1. Страница по умолчанию

После запуска приложения пользователь перенаправляется на страницу (Рисунок 1)

Add New Corporate Employee

System String ID:
e.g., emp_05

Full Name:
Ivan Ivanov

Corporate Email:
ivan@org.ru

Age (Optional):
Min value 18

Corporate Department:
Information Technology (IT) ▼

Corporate Position / Title:
e.g., Senior Engineer

Official Hire Date:
dd/mm/yyyy

Employment Operational Status:
Active On-Site ▼

Register Record

Рисунок 2. Страница администратора

После входа в систему в качестве администратора путем выбора соответствующей опции открывается главная страница приложения. Здесь администратор может ввести данные сотрудников, такие как идентификатор, полное имя, возраст, должность, отдел и статус.

Active Working Session: Administrator							
Detailed Corporate Directory Matrix							
ID	Full Name	Department	Position	Email	Age	Hire Date	Status
12	Binh Ng	IT	Engineer	binhng@org.ru	21	2026-06-12	• Active
13	Vo Vu	HR	HR	vv@org.ru	—	2026-06-04	• Remote
14	Huy Do	Sales	Salesman	hd@org.ru	—	2026-04-03	• On Leave
15	An Hu	IT	Dev	ah@org.ru	24	2025-12-14	• Remote

Рисунок 3. Таблица информации о сотрудниках

(Рисунок 3) отображает таблицу, синхронизированную с базой данных PostgreSQL через сервер. Для каждого сотрудника отображаются его имя, адрес электронной почты, возраст, должность и отдел. На этой панели администратор может легко просматривать информацию и статус сотрудников.

Official Hire Date:

dd/mm/yyyy

Employment Operational Status:

Active On-Site

Register Record

✓

Employee Binh Ng added successfully!

Рисунок 4. Отзывы о сайте

Если информация неверна, она не будет передана; если все поля верны, отобразится сообщение ✓ Сотрудник Binh Ng успешно добавлен!

The screenshot shows a web browser window with the URL 'localhost:5176'. The page title is 'OrgSphere Portal'. In the top right corner, there are three elements: a green button labeled 'Informations', a blue button labeled 'Mode: Dark', and a dropdown menu labeled 'Role: Employee'. Below these, the text 'Active Working Session: Employee' is displayed. Underneath, the title 'Detailed Corporate Directory Matrix' is shown. The main content is a table with 8 columns: ID, Full Name, Department, Position, Email, Age, Hire Date, and Status. The table contains 4 rows of employee data.

ID	Full Name	Department	Position	Email	Age	Hire Date	Status
12	Binh Ng	IT	Engineer	binhng@org.ru	21	2026-06-12	Active
13	Vo Vu	HR	HR	vv@org.ru	—	2026-06-04	Remote
14	Huy Do	Sales	Salesman	hd@org.ru	—	2026-04-03	On Leave
15	An Hu	IT	Dev	ah@org.ru	24	2025-12-14	Remote

Рисунок 5. Таблица данных сотрудника.

Для штатных сотрудников предусмотрен упрощенный рабочий интерфейс (рисунок 5). После входа в систему они видят личную панель управления со списком статусов сотрудников.

Взаимодействие между клиентом и сервером NestJS осуществляется асинхронно с использованием сетевого клиента Axios через HTTP-запросы (GET, POST, PUT, DELETE). Для обмена данными используется стандартный формат JSON. Как только сервер возвращает успешный ответ, код React JavaScript мгновенно обновляет состояние компонента с помощью встроенных хуков, а клиентское приложение обеспечивает удобный и отзывчивый пользовательский интерфейс с системой корпоративного управления OrgSphere. Оно предоставляет удобные инструменты для мониторинга ресурсов, управления персоналом и осуществления обмена данными.

ЗАКЛЮЧЕНИЕ

В рамках настоящей курсовой работы было спроектировано и реализовано клиент-серверное веб-приложение «OrgSphere», предназначенное для автоматизации внутренних процессов корпоративного управления, менеджмента персонала и распределения задач ИТ-предприятия.

В ходе выполнения работы успешно решены все поставленные задачи. Спроектирована и развернута реляционная база данных PostgreSQL, структура которой приведена к третьей нормальной форме (3NF). На стороне СУБД разработаны развитые программируемые объекты на языке PL/pgSQL: аналитические представления для контроля нагрузки, транзакционные хранимые процедуры для безопасного перевода сотрудников, а также автоматические триггеры, реализующие реактивную логику (каскадное обновление статусов проектов и валидацию возраста на уровне ядра БД).

Серверная часть приложения была разработана на базе фреймворка NestJS с использованием языка TypeScript. В ходе реализации была создана гибридная модель обработки данных, сочетающая in-memory кэширование реестра пользователей в оперативной памяти сервера с персистентным сохранением связанных сущностей через TypeORM. Дополнительно была внедрена строгая DTO-валидация входящих JSON-потоков и глобальная фильтрация исключений. Клиентская часть приложения реализована в виде SPA-интерфейса на базе библиотеки React с использованием хуков управления состоянием (useState, useEffect) и асинхронного транспорта Axios.

Реализованный функционал включает полную поддержку сквозных CRUD-операций для ключевых сущностей инфраструктуры организации, интерактивные формы управления с живой перерисовкой компонентов без перезагрузки страниц, автоматический контроль целостности бизнес-показателей и разграничение прав доступа.

Практическая значимость работы заключается в создании готового архитектурного шаблона, позволяющего автоматизировать распределение ресурсов, исключить ошибки ручного учета и повысить транзакционную консистентность данных. Внедрение подобного решения в деятельность современных ИТ-компаний способствует оптимизации менеджмента задач и сокращению времени отклика интерфейсов в условиях повышенных нагрузок. Таким образом, разработанное приложение полностью соответствует техническим требованиям и подтверждает теоретические знания, полученные в рамках изучения дисциплины.

БИБЛИОГРАФИЧЕСКИЙ СПИСОК

1. Голицына, О. Л. Базы данных : учебное пособие для вузов / О. Л. Голицына, Н. В. Максимов, И. И. Попов. – 4-е изд., перераб. и доп. – Москва : ФОРУМ : ИНФРА-М, 2025. – 400 с. – ISBN 978-5-0009-1601-8.
2. Холден, Б. Разработка веб-приложений с использованием NestJS и TypeScript / Б. Холден. – Пер. с англ. – Санкт-Петербург : Питер, 2024. – 320 с. – ISBN 978-5-4461-2103-4.
3. Официальное руководство по фреймворку NestJS: Архитектура, Контроллеры, Сервисы и Инъекция зависимостей.
4. Документация СУБД PostgreSQL: Руководство по языку программирования PL/pgSQL, созданию триггеров и хранимых процедур.
5. Официальный учебник по библиотеке React: Разработка функциональных компонентов, использование хуков `useState` и `useEffect`.
6. Спецификация TypeORM: Объектно-реляционное картирование и управление реляционными транзакциями в Node.js приложениях.
7. ГОСТ 7.32-2017. Система стандартов по информации, библиотечному и издательскому делу. Отчет о научно-исследовательской работе. Структура и правила оформления. – Москва : Стандартинформ, 2017.

ПРИЛОЖЕНИЕ А: Полный DDL-скрипт развертывания реляционной базы данных

Приложение А.1: Создание физических таблиц и ограничений целостности

```
CREATE TABLE departments (  
    id SERIAL PRIMARY KEY,  
    name VARCHAR(150) NOT NULL UNIQUE,  
    budget NUMERIC(12, 2) NOT NULL DEFAULT 0.00  
);  
CREATE TABLE users (  
    id SERIAL PRIMARY KEY,  
    name VARCHAR(100) NOT NULL,  
    email VARCHAR(150) NOT NULL UNIQUE,  
    age INTEGER NOT NULL,  
    role_id INTEGER NOT NULL DEFAULT 1,  
    department_id INTEGER,  
    CONSTRAINT fk_user_department FOREIGN KEY (department_id)  
        REFERENCES departments(id) ON DELETE RESTRICT ON UPDATE  
CASCADE  
);  
CREATE TABLE projects (  
    id SERIAL PRIMARY KEY,  
    title VARCHAR(200) NOT NULL UNIQUE,  
    deadline DATE NOT NULL,  
    status VARCHAR(50) NOT NULL DEFAULT 'Pending'  
);  
CREATE TABLE tasks (  
    id SERIAL PRIMARY KEY,  
    description TEXT NOT NULL,  
    project_id INTEGER NOT NULL,  
    assigned_to INTEGER,  
    status VARCHAR(50) NOT NULL DEFAULT 'Pending',  
    CONSTRAINT fk_task_project FOREIGN KEY (project_id)  
        REFERENCES projects(id) ON DELETE CASCADE ON UPDATE  
CASCADE,  
    CONSTRAINT fk_task_user FOREIGN KEY (assigned_to)  
        REFERENCES users(id) ON DELETE SET NULL ON UPDATE  
CASCADE  
);  
CREATE TABLE audit_budget_log (  
    id SERIAL PRIMARY KEY,  
    department_id INTEGER,  
    old_budget NUMERIC(12, 2),  
    new_budget NUMERIC(12, 2),  
    changed_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP  
);
```


Приложение А.2: Создание реляционных аналитических представлений (Views)

```
CREATE OR REPLACE VIEW v_employee_workload AS
SELECT
    u.id AS employee_id,
    u.name AS employee_name,
    d.name AS department_name,
    COUNT(t.id) AS active_tasks_count
FROM users u
LEFT JOIN departments d ON u.department_id = d.id
LEFT JOIN tasks t ON t.assigned_to = u.id AND t.status !=
'Completed'
GROUP BY u.id, u.name, d.name;

CREATE OR REPLACE VIEW v_department_analytics AS
SELECT
    d.id AS department_id,
    d.name AS department_name,
    d.budget AS total_budget,
    COUNT(DISTINCT u.id) AS total_employees
FROM departments d
LEFT JOIN users u ON u.department_id = d.id
GROUP BY d.id, d.name, d.budget;
```

Приложение А.3: Имплементация триггеров бизнес-логики на языке PL/pgSQL

```
CREATE OR REPLACE FUNCTION fn_trg_prevent_underage_hiring()
RETURNS TRIGGER AS $$
BEGIN
    IF NEW.age < 18 THEN
        RAISE EXCEPTION 'Критическая ошибка валидации ядра:
Сотрудник % не может быть зарегистрирован. Минимальный возраст
– 18 лет.', NEW.name
        USING ERRCODE = 'HEX01';
    END IF;
    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER trg_prevent_underage_hiring
BEFORE INSERT OR UPDATE ON users
FOR EACH ROW EXECUTE FUNCTION fn_trg_prevent_underage_hiring();

CREATE OR REPLACE FUNCTION fn_trg_auto_update_project_status()
```

```

RETURNS TRIGGER AS $$
DECLARE
    incomplete_tasks_count INT;
BEGIN
    SELECT COUNT(*) INTO incomplete_tasks_count
    FROM tasks
    WHERE project_id = NEW.project_id AND status !=
'Completed';

    IF incomplete_tasks_count = 0 THEN
        UPDATE projects SET status = 'Completed' WHERE id =
NEW.project_id;
    END IF;

    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER trg_auto_update_project_status
AFTER UPDATE OF status ON tasks
FOR EACH ROW EXECUTE FUNCTION
fn_trg_auto_update_project_status();

CREATE OR REPLACE FUNCTION fn_trg_audit_budget_changes()
RETURNS TRIGGER AS $$
BEGIN
    IF OLD.budget IS DISTINCT FROM NEW.budget THEN
        INSERT INTO audit_budget_log (department_id,
old_budget, new_budget)
        VALUES (NOT NULL, OLD.id, OLD.budget, NEW.budget);
    END IF;
    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER trg_audit_budget_changes
AFTER UPDATE OF budget ON departments
FOR EACH ROW EXECUTE FUNCTION fn_trg_audit_budget_changes();

```

Приложение А.4: Имплементация хранимых транзакционных процедур

```

CREATE OR REPLACE PROCEDURE transfer_employee(
    IN p_emp_id INTEGER,
    IN p_new_dept_id INTEGER
)
AS $$

```

```

BEGIN
    IF NOT EXISTS (SELECT 1 FROM departments WHERE id =
p_new_dept_id) THEN
        RAISE EXCEPTION 'Целевой департамент с ID % не
зарегистрирован в системе.', p_new_dept_id;
    END IF;

    -- Атомарное обновление запис
UPDATE users
SET department_id = p_new_dept_id
WHERE id = p_emp_id;

    COMMIT;
END;
$$ LANGUAGE plpgsql;

```

ПРИЛОЖЕНИЕ Б: Исходный код серверных компонентов NestJS (Бизнес-логика)

Приложение Б.1: Описание ORM-сущности сотрудника (user.entity.ts)

```

import { Entity, PrimaryGeneratedColumn, Column, ManyToOne,
JoinColumn } from 'typeorm';
import { Department } from '../department.entity';

@Entity('users')
export class User {
    @PrimaryGeneratedColumn()
    id: number;

    @Column({ type: 'varchar', length: 100 })
    name: string;

    @Column({ type: 'varchar', length: 150, unique: true })
    email: string;

    @Column({ type: 'integer' })
    age: number;

    @Column({ type: 'integer', name: 'role_id', default: 1 })
    roleId: number;

    @Column({ type: 'integer', name: 'department_id', nullable:
true })
    departmentId: number;

```

```

@ManyToOne(() => Department, { onDelete: 'RESTRICT' })
@JoinColumn({ name: 'department_id' })
department: Department;
}

```

Приложение Б.2: Класс валидации входных параметров DTO

```

import { IsString, IsEmail, IsInt, Min, IsOptional } from
'class-validator';

export class CreateUserDto {
  @IsString({ message: 'Имя сотрудника должно быть текстовой
строкой' })
  readonly name: string;

  @IsEmail({}, { message: 'Некорректный формат адреса
электронной почты' })
  readonly email: string;

  @IsInt({ message: 'Возраст должен быть целым числом' })
  @Min(18, { message: 'Нарушение трудового законодательства:
минимальный возраст — 18 лет' })
  readonly age: number;

  @IsOptional()
  @IsInt()
  readonly departmentId?: number;
}

```

Приложение Б.3: Серверный компонент бизнес-логики (users.service.ts с In-Memory репозиторием)

```

import { Injectable, BadRequestException,
InternalServerErrorException } from '@nestjs/common';
import { InjectRepository } from '@nestjs/typeorm';
import { Repository } from 'typeorm';

```

```

import { User } from './user.entity';
import { CreateUserDto } from './create-user.dto';

@Injectable()
export class UsersService {
  private usersMemoryRepository: User[] = [];

  constructor(
    @InjectRepository(User)
    private readonly usersDbRepository: Repository<User>,
  ) {}

  async findAllFromMemory(): Promise<User[]> {
    return this.usersMemoryRepository;
  }

  async createUser(dto: CreateUserDto): Promise<User> {
    const newUser = this.usersDbRepository.create(dto);
    let savedDbUser: User;

    try {
      savedDbUser = await this.usersDbRepository.save(newUser);
    } catch (error) {
      throw new BadRequestException(`Ошибка СУБД PostgreSQL:
${error.message}`);
    }

    this.usersMemoryRepository.push(savedDbUser);

    return savedDbUser;
  }

  async deleteUser(id: number): Promise<{ success: boolean }> {
    const numericId = Number(id);

    await this.usersDbRepository.delete(numericId);

    this.usersMemoryRepository =
this.usersMemoryRepository.filter(
  (user) => user.id !== numericId,
);

    return { success: true };
  }
}

```

Приложение Б.4 — Серверный контроллер маршрутизации API (users.controller.ts)

```
import { Controller, Get, Post, Delete, Body, Param,
ParseIntPipe } from '@nestjs/common';
import { UsersService } from '../users.service';
import { CreateUserDto } from '../create-user.dto';
import { User } from '../user.entity';

@Controller('api/users')
export class UsersController {
  constructor(private readonly usersService: UsersService) {}

  @Get()
  async getAllUsers(): Promise<User[]> {
    // Обращение к Low-Latency слою оперативной памяти
    return this.usersService.findAllFromMemory();
  }

  @Post()
  async createUser(@Body() createUserDto: CreateUserDto):
Promise<User> {
    // Входная точка создания, защищенная ValidationPipe
    return this.usersService.createUser(createUserDto);
  }

  @Delete('/:id')
  async removeUser(@Param('id', ParseIntPipe) id: number) {
    return this.usersService.deleteUser(id);
  }
}
```

ПРИЛОЖЕНИЕ В: Исходный код клиентских компонентов React (Пользовательский интерфейс)

Приложение В.1: Главный управляющий компонент интерфейса (App.jsx)

```
import React, { useState } from 'react';
import UsersManager from '../components/UsersManager';
import ProjectsManager from '../components/ProjectsManager';
import './styles.css';

function App() {
```

```

const [activeTab, setActiveTab] = useState('users');

return (
  <div className="app-container">
    {/* Боковая панель навигации Сайдбар */}
    <aside className="sidebar">
      <div className="brand">OrgSphere v2026</div>
      <div className="role-badge">Студент: Бу Тхань
Бинь</div>
      <nav className="nav-menu">
        <button
          className={activeTab === 'users' ? 'nav-btn active'
: 'nav-btn'}
          onClick={() => setActiveTab('users')}
        >
          Сотрудники (In-Memory/DB)
        </button>
        <button
          className={activeTab === 'projects' ? 'nav-btn
active' : 'nav-btn'}
          onClick={() => setActiveTab('projects')}
        >
          Проекты и Задачи (Trigger)
        </button>
      </nav>
    </aside>

    {/* Центральная контентная панель */}
    <main className="content-panel">
      {activeTab === 'users' && <UsersManager />}
      {activeTab === 'projects' && <ProjectsManager />}
    </main>
  </div>
);
}

export default App;

```

Приложение В.2: Модуль реактивного управления сотрудниками (UsersManager.jsx)

```

import React, { useState, useEffect } from 'react';
import axios from 'axios';

```

```

function UsersManager() {
  const [users, setUsers] = useState([]);
  const [formData, setFormData] = useState({ name: '', email:
'', age: '', departmentId: '' });
  const [errorMessage, setErrorMessage] = useState('');

  useEffect(() => {
    fetchUsers();
  }, []);

  const fetchUsers = async () => {
    try {
      const response = await axios.get('/api/users');
      setUsers(response.data);
    } catch (error) {
      setErrorMessage('Не удалось загрузить реестр сотрудников
из оперативной памяти бэкенда.');
```



```

const handleDelete = async (id) => {
  if (window.confirm(`Вы уверены, что хотите удалить
сотрудника с ID ${id}?`)) {
    try {
      await axios.delete(`/api/users/${id}`);
      setUsers(users.filter(user => user.id !== id));
    } catch (error) {
      alert('Ошибка при удалении записи сотрудника.');
```

```

        <tr key={user.id}>
          <td>{user.id}</td>
          <td><strong>{user.name}</strong></td>
          <td>{user.email}</td>
          <td>{user.age} лет</td>
          <td>
            <button onClick={() => handleDelete(user.id)}
className="btn-small btn-danger">
              Удалить
            </button>
          </td>
        </tr>
      </tbody>
    </table>
  </div>
);
}

export default UsersManager;

```

